Orbit Wars — RL Formulation and GRPO Training Math

Native C++/LibTorch stack

June 5, 2026

This document specifies the policy, the reward, and the *Group Relative Policy Optimization* (GRPO) update used by the native trainer. Symbols map to `native/rl/config.hpp` and `native/rl/model/`.

Contents

1 Problem setup

Orbit Wars is an N-player symmetric Markov game with $N \in \{2, 4\}$. The engine sets $N = |\text{state}|$ (`num_agents`) and assigns N home planets from a 4-fold-symmetric quadrant group ($N = 2$: two

opposite quadrants; $N = 4$: all four). Each turn all players *simultaneously* issue a set of fleet *launches*; the deterministic engine then moves fleets, resolves collisions/combat, rotates planets, produces ships, and (on scheduled steps) spawns comets. Scoring is a **free-for-all**: at termination player i's score is its total ship count ships_i (planets + fleets), and the seat(s) with the maximum score receive reward $+1$, all others -1 (so $N = 4$ is winner-take-all among four individuals — not teams).

Training scope of this document. We currently train the **two-player special case** ($N = 2$: ego $p = 0$ against one opponent in the opposite quadrant). The native world generator places exactly two homes; the shaped reward (Section 5) uses the zero-sum margin $\text{ships}_0 - \text{ships}_1$; and each group plays a single opponent (scripted, or a frozen snapshot, Section 8). The policy (Section 3) and the GRPO update (Section 7) are *N-agnostic*; lifting to 4-player FFA requires only (i) 4-home world generation, (ii) replacing the zero-sum margin with the FFA return (max-score seat $+1$ else -1, or a rank-based shaping), and (iii) seating ≥ 1 extra opponents per game. This 2p→4p gap is an open modeling item, revisited in Section 5.

Continuous, per-planet action space. A turn's action is *factored over the E planet slots* of the observation ($E = \texttt{max\_entities} = 40$). Each slot e emits a fixed block of $3K$ *continuous* controls ($K = 5$ fleet slots, three scalars each), so the whole action is a 40×15 matrix

$$a \in [0,1]^{E \times 3K}, \qquad E = 40,\ K = 5. \tag{1}$$

Fleet slot k of planet e is a triple $(d^x_{e,k}, d^y_{e,k}, \phi_{e,k}) \in [0,1]^3$: a *direction vector* (d^x, d^y), recentred to $[-1, 1]$ as $2d - 1$ and mapped to a launch angle $\theta_{e,k} = \text{atan2}(2d^y_{e,k} - 1,\ 2d^x_{e,k} - 1)$ (0–360°, continuous — no bins), and a *ship fraction* $\phi_{e,k}$ mapped to 0–100% of the launching planet's current ships. The direction-vector parameterization (replacing an earlier scalar heading $\theta = 2\pi\alpha$) makes the heading **linear in relative position** — attention over the planet tokens (Section 3) can produce a target-minus-self vector directly, whereas a linear head cannot turn a relative position into the angle atan2$(\cdot)$ a scalar α would require. A planet may thus dispatch up to $K = 5$ fleets in one step, each to an independently chosen heading and size; the engine processes a planet's K launches in slot order, deducting ships *sequentially*. There is no explicit no-op class: a fleet with $\phi \approx 0$ (below the activation threshold τ_{act}) simply launches nothing.

No masking — the legal action space is *learned* from penalties. The action head is **not** masked by which planet slots exist or which the ego owns. The policy emits the full 40×15 block every step and learns the dynamic legal subset from the invalid-dispatch penalty of Section 5: the desired output for a non-existent slot is the zero action; dispatching from a planet the ego does not own (or that has 0 ships) is penalized; and *over-dispatch* (committing more than 100% of a planet's ships across its K fleets) is penalized — in all cases, "everywhere." This continuous, target-free space **supersedes the earlier discrete** target-mode / angle-bin categorical actions; continuous headings restore full aiming precision without the 16-bin aiming barrier, and without the target-pointer machinery.

2 Observation encoding

For ego player p and state s, the encoder (`native/core/encode.hpp`) emits

- entity features $X \in \mathbb{R}^{E\times F}$ ($F = 32$): per planet — 11 *body* features (position x, y; orbital velocity magnitude; clockwise/counter-clockwise sign; radius; production; ships; ownership code [0 neutral, 1 ego, 2–4 enemy seat]; distance-to-center; a *comet flag* $\mathbb{1}$[e is a comet]; and the action mask u_e), plus a 21-feature *incoming-threat block* (below). *Radius* and *production* are redundant for normal planets ($r = 1+\ln(\text{prod})$), but we feed both so the net need neither invert the ln nor re-derive the reward-defining quantity. A *stationary* body has velocity 0 and no rotation sign, but an *orbiting* planet and a *comet* both move, so velocity alone cannot separate them — hence the explicit comet flag, which also marks where the radius law breaks (a comet carries the fixed `COMET_PRODUCTION`= 1 at an unrelated radius). The comet's remaining lifetime is **deliberately hidden by the engine** (agents may not reconstruct the spawn/expiry schedule), so a binary flag is the only observable comet signal;
- board-summary globals $g \in \mathbb{R}^G$ ($G = 10$): a *separate* small input (step/phase, board-wide ship and production margins, planet counts, angular velocity, ...) — **not** replicated into each planet row, but embedded once and broadcast to the action heads (Section 3);
- entity mask $m \in \{0,1\}^E$: $m_e = 1$ iff slot e is a real entity;
- action mask $u \in \{0,1\}^E$: $u_e = 1$ iff e is an ego-owned planet with > 0 ships (legally able to launch). It is *both* a per-planet input feature and the signal used to *decode* launches and score the invalid-dispatch penalty (Section 5); *neither m nor u masks the actor* — the policy learns the legal subset from the penalty, not a hard gate.

This game is a **free-for-all**: there are no allies; an inbound fleet is either the ego's own (a reinforcement) or an enemy's, distinguished by the owner feature in the threat block.

Incoming-fleet threat block (per planet, baked into the row). A purely *reactive* policy cannot defend if it cannot see what is coming. For each planet we therefore bake a compact list of *its own* inbound fleets directly into that planet's feature row. First we project every in-flight fleet to the planet it will hit, and when. A fleet at position o with unit heading $\hat{d} = (\cos\alpha, \sin\alpha)$ and ship count ν moves at speed

$$v(\nu) = \min\Big(v_{\max},\ 1 + (v_{\max}-1)\big(\tfrac{\ln\max(1,\nu)}{\ln 1000}\big)^{1.5}\Big), \qquad v_{\max} = 6, \tag{2}$$

matching the engine's fleet dynamics. For a planet at c with radius ϱ, the ray $o + t\hat{d}$ enters the planet disk at

$$t_{\text{ca}} = -\langle o-c, \hat{d}\rangle, \quad p^2 = \|o-c\|^2 - t_{\text{ca}}^2, \quad t_{\text{hit}} = t_{\text{ca}} - \sqrt{\varrho^2 - p^2} \ \ (\text{if } t_{\text{ca}} \geq 0,\ p^2 \leq \varrho^2). \tag{3}$$

The fleet is assigned to the planet with the smallest $t_{\text{hit}} \geq 0$ (dropped if the ray first crosses the central sun — a *fixed* disk of radius SUN_RADIUS = 10, constant all game — or hits nothing), with ETA $\tau = t_{\text{hit}}/v(\nu)$ in ticks.

Two ranked views per planet (tuneable counts). Among the fleets assigned to a planet we keep two short lists: the `N_SOON`= 2 with the smallest ETA (most *imminent*) and the `N_BIG`= 5 with the largest ship count (most *dangerous*); the lists may overlap. Each of the 7 slots contributes 3 features

$$\big[\ \text{sgn},\ \ \tau,\ \ \widetilde{\nu}\ \big], \qquad \widetilde{\nu} = \frac{\ln(1+\nu)}{\ln 1000}, \tag{4}$$

i.e. *who it belongs to* (sgn = +1 ego reinforcement / −1 enemy), *arrival time* as the raw ETA τ in ticks, and *ship count* $\widetilde{\nu}$. Empty slots are all zero — sgn = 0 there already marks "no fleet," so $\tau = 0$ is unambiguous — giving the row's $7 \times 3 = 21$ threat features. Because the list is attached

to the destination planet's *own* row, that planet directly sees "what is coming at *me* — how soon, how big, and from whom" — local defensive awareness with no attention and no separate fleet tokens. The projection ignores planet rotation but is computed *identically* in C++ and Python (parity-tested bit-for-bit), so the natively trained policy behaves identically when served from pure Python.

3 Policy network π_θ

The network (`native/rl/model/policy_net.hpp`) maps the per-planet entity matrix $X \in \mathbb{R}^{E\times F}$ (body + incoming-threat features, Section 2) to a per-planet block of $3K$ continuous action parameters. The $G = 10$ board-summary globals enter through a *small separate path*: a one-layer embedding $g' = \mathrm{ReLU}(W_g\, g + b_g) \in \mathbb{R}^{d_g}$ ($d_g = 32$), broadcast to every planet and concatenated at the action heads — so the trunk itself stays purely per-planet (per-row). It has **no value head** (GRPO needs no critic) and **no pointer/target head** (the action is continuous control). Let $d =$ `hidden`.

Single-stream trunk (the two variants under comparison). Every planet row is processed independently and identically (weights shared across rows): an input projection $h_e^{(0)} = W_{\mathrm{proj}} x_e + b_{\mathrm{proj}}$, then a stack of residual blocks. We compare two trunks via `ModelConfig::use_glu`, with everything downstream identical:

(A) GLU + stacked MLP: one residual GLU block $\mathcal{B}$ (Eq. 6, Fig. 2) then n_{res}=2 residual MLP blocks $\mathcal{M}$;
(B) stacked MLP only: n_{res}+1 residual MLP blocks (the GLU is replaced by a residual MLP block, matching depth) — no multiplicative gating.

A residual MLP block is $\mathcal{M}(z) = z + W_b\ \mathrm{ReLU}(W_a z + b_a) + b_b$, so

$$h_e = \big(\underbrace{\mathcal{M}\circ\mathcal{M}}_{n_{\mathrm{res}}}[\circ\,\mathcal{B}]\big)\big(h_e^{(0)}\big) \in \mathbb{R}^d, \qquad e = 1,\dots,E, \tag{5}$$

(the bracketed $\mathcal{B}$ present only in variant A). The trunk has no mask-aware mean-pool and no cross-row mixing — each planet is processed independently from its own body + threat features; the only board-wide signal is the small broadcast embedding g' added at the heads.

Residual GLU block. For an input $z \in \mathbb{R}^d$, with three $d \times d$ weight matrices (gate W_g, value W_v, output W_o),

$$\mathrm{GLU}(z) = \big(W_v z + b_v\big) \odot \sigma\big(W_g z + b_g\big), \qquad \mathcal{B}(z) = z + W_o\,\mathrm{GLU}(z) + b_o \ \in \mathbb{R}^d, \tag{6}$$

where σ is the logistic sigmoid and $\odot$ is elementwise. The sigmoid gate lets the block pass, attenuate, or suppress each feature channel conditionally (e.g. amplify the incoming-threat channels of Section 2 when a planet is under attack); the residual keeps optimization easy. It is applied per planet row and shares weights across rows just like the rest of the trunk. Whether this gating earns its keep over plain stacked MLPs (variant A vs. B) is exactly the comparison this experiment is set up to answer.

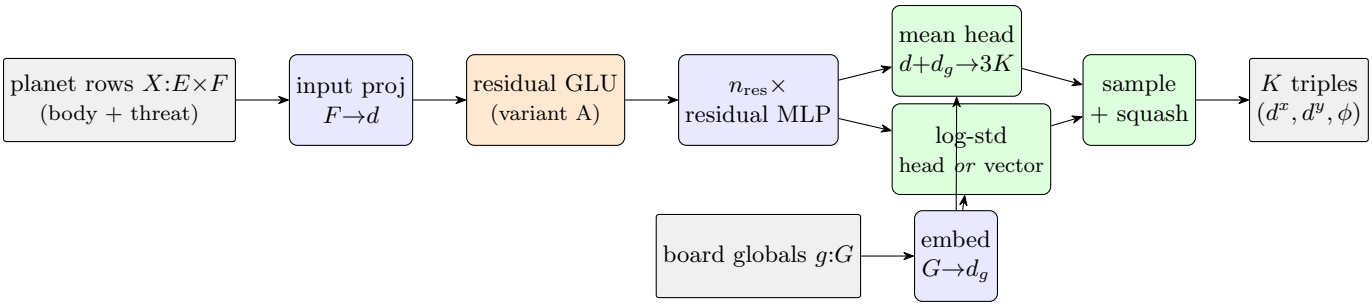

Figure 1: Policy network data flow (continuous action). Each planet row (body + incoming-threat features) is projected and passed through the shared per-row trunk: variant A inserts one residual GLU block (Fig. 2) before the n_{res} residual MLP blocks; variant B uses residual MLP blocks only. The G board globals are embedded separately (g', dim d_g) and broadcast to the heads. A mean head maps $[h_e; g']$ to the $3K$ means μ_e; the $3K$ log-stds ς_e come from *either* a second head (state-dependent) *or* a shared learnable vector (state-independent) — a switchable A/B. Together they define a squashed Gaussian (Section 4) whose samples are the K fleet triples (d^x, d^y, ϕ) for planet e. No pooling, no cross-row mixing, no value or pointer head.

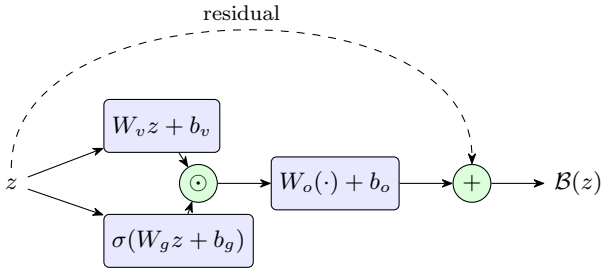

Figure 2: The residual GLU block $\mathcal{B}$ (Eq. 6): a value branch $W_v z$ and a sigmoid gate $\sigma(W_g z)$ are multiplied elementwise ($\odot$), projected by W_o, and added back to the input (the residual skip). In trunk variant A one such block (its own $d \times d$ weights) follows the input projection, before the residual MLP stack.

Continuous action heads (mean, and a switchable log-std). A *mean* head reads each planet token, $\mu_e \in \mathbb{R}^{3K}$. The log-std ς_e is supplied in one of **two switchable ways** (`ModelConfig::std_state_dependent`) kept as an explicit A/B because they trade off expressiveness against training stability:

State-dependent (a head): $\varsigma_e = W_\varsigma h_e + b_\varsigma \in \mathbb{R}^{3K}$ — the spread depends on the board, so the policy explores where the board is open/uncertain and commits (small σ) where the move is forced (e.g. a clear defensive launch). More expressive, but can destabilize (it may drive $\sigma \to 0$ in "confident" states and stop exploring).

State-independent (a vector): $\varsigma_e \equiv \varsigma \in \mathbb{R}^{3K}$, one learnable vector shared across all planets and states — the standard, stable PPO default.

Either way, $\sigma_e = \exp\big(\text{clip}(\varsigma_e, \varsigma_{\min}, \varsigma_{\max})\big)$, the clip keeping σ from collapsing to 0 or exploding; the mean head and trunk are identical across the two, only the source of ς differs, and ς is trained by the same surrogate+entropy gradients (Section 4). Each head emits $3K = 15$ numbers — the per-component parameters of the K fleet triples (d^x, d^y, ϕ); sampling and the $[0, 1]$ squash are in Section 4, decoding into engine launches in Section 4. Plain linear heads suffice because the action

is continuous control rather than a discrete target ranking, so the pointer actor of the previous design is retired together with target mode.

Layer inventory ($d = 128$, variant A). All layers are `nn.Linear`. The trunk is the input projection, one residual GLU block (variant A only), and n_{res}=2 residual MLP blocks; then the small board embedding and the two action heads.

Module	Layers (name: shape)	Count
Input projection	`proj`:$F \to d$	1
Residual GLU (variant A)	`glu_{gate,val,out}`:$d \to d$	3
Residual MLP $\times n_{\text{res}}$	`r{0,1}a`, `r{0,1}b`:$d \to d$	4
Board-globals embed	`g_embed`:$G \to d_g$	1
Mean head	`mu_head`:$d+d_g \to 3K$	1
Log-std (switchable)	`logstd_head`:$d+d_g \to 3K$ *or* vector ς:$3K$	1
Trainable total (no critic, no pointer — GRPO)		**11**

Variant B replaces the 3-layer GLU block with one 2-layer residual MLP block (i.e. 3 residual MLP blocks, no gating). With F=30 (9 body + 21 threat), G=10, d_g=32, d=128, K=5 this is ≈ 0.12M parameters (A) / ≈ 0.10M (B). The pure-Python serving twin (`orbit_wars_rl/agents/ppo_policy.py`) must mirror this trunk, board path, and heads bit-for-bit for the Kaggle submission to reproduce the natively trained policy; it carries no critic.

4 Action distribution

The policy is a **squashed diagonal Gaussian** over the $E \times 3K$ continuous controls. For each component $a_{e,k,j}$ ($j \in \{d^x, d^y, \phi\}$, the two direction components / ship fraction of fleet k of planet e) the network emits a pre-squash mean $\mu_{e,k,j}$ *and* a log-std $\varsigma_{e,k,j}$ — the mean always per-planet, the log-std either per-planet (state-dependent) or a shared vector $\varsigma_{k,j}$ (state-independent), switchable (Section 3); we draw a latent $z_{e,k,j} \sim \mathcal{N}(\mu_{e,k,j}, \sigma^2_{e,k,j})$ with $\sigma_{e,k,j} = e^{\varsigma_{e,k,j}}$ and squash it into the unit interval with the logistic $\varphi(z) = 1/(1+e^{-z})$:

$$a_{e,k,j} = \varphi(z_{e,k,j}) \in (0,1). \tag{7}$$

The components are conditionally independent given s, so the joint log-probability is a sum that *includes* the change-of-variables (squash Jacobian) term:

$$\log \pi_\theta(a \mid s) = \sum_{e=1}^{E} \sum_{k=1}^{K} \sum_{j \in \{d^x, d^y, \phi\}} \Big[\log \mathcal{N}\big(z_{e,k,j};\, \mu_{e,k,j}, \sigma^2_{e,k,j}\big) - \log\big(a_{e,k,j}(1-a_{e,k,j})\big) \Big], \tag{8}$$

where $\log\big(a(1-a)\big) = \log|\mathrm{d}a/\mathrm{d}z|$. The entropy bonus uses the closed-form latent-Gaussian differential entropy, summed over components (a constant apart, it is just the total log-std),

$$\mathcal{H}\big[\pi_\theta(\cdot \mid s)\big] = \sum_{e,k,j} \Big(\varsigma_{e,k,j} + \tfrac{1}{2} \log 2\pi e \Big), \tag{9}$$

so the bonus regularizes the exploration scale directly. Unlike the previous discrete policy, the sums run over *all* E planet slots, not only real entities: the invalid-dispatch penalty (Section 5) — not a mask — is what drives the controls on non-existent / unowned slots toward "no launch." At evaluation the policy is greedy, $a_{e,k,j} = \varphi(\mu_{e,k,j})$ (no sampling). *(Remark: the Gaussian is*

over the Cartesian direction components (d^x, d^y), not a circular angle, so there is no $0/360^\circ$ wrap discontinuity — the earlier scalar-heading parameterization had one; emitting a direction vector and decoding $\theta = \text{atan2}(\cdot)$ removes it, at the cost of a benign indeterminacy only at the exact centre $d^x = d^y = \frac{1}{2}$.)

How the mean and log-std heads are learned (no separate target). Neither head has labels; both are trained *only* by the GRPO surrogate (Section 7) and the entropy bonus, through the dependence of $\log \pi_\theta$ on them. Per component, with the standardized sample $u = (z - \mu)/\sigma$,

$$\frac{\partial \log \mathcal{N}}{\partial \mu} = \frac{u}{\sigma}, \qquad \frac{\partial \log \mathcal{N}}{\partial \varsigma} = u^2 - 1 \quad (\text{since } \sigma = e^\varsigma). \tag{10}$$

A policy-gradient step moves each parameter along $\hat{A} \cdot \partial \log \pi_\theta / \partial(\cdot)$ (plus the entropy term). For the **mean**: a sampled action that beat its group baseline ($\hat{A} > 0$) pulls μ toward the action taken, one that underperformed pushes it away. For the **log-std**: the factor $u^2 - 1$ is positive when the draw landed *far* from the mean ($|z - \mu| > \sigma$) and negative when *close*, so a far-out action that *helped* ($\hat{A} > 0$) **widens** σ (explore more there), a far-out action that *hurt* **narrows** it (exploit), and a near-mean action that helped also narrows it (commit to the good mean). The entropy bonus adds a constant $+c_\mathcal{H}$ to the log-std gradient ($\partial \mathcal{H}/\partial \varsigma = 1$), a steady upward pressure against premature collapse of σ. Because $\varsigma_e = W_\varsigma h_e$ is produced by the log-std head, these gradients flow on into W_ς and the shared trunk — that is how the network learns *where* on the board to be exploratory versus decisive. (In the state-independent variant the same gradients instead accumulate over all slots into the one shared ς, tuning a single global exploration scale.) Adam applies the update; ς is clipped to $[\varsigma_{\min}, \varsigma_{\max}]$ for numerical safety.

Decode: controls → engine launches. For each ego-owned planet e (ownership $u_e = 1$, ships $S_e > 0$), the K fleet triples are emitted in slot order; fleet k launches $n_{e,k} = \lfloor \phi_{e,k} S_e \rfloor$ ships at angle $\theta_{e,k} = \text{atan2}(2d^y_{e,k} - 1,\, 2d^x_{e,k} - 1)$, and the engine deducts them *sequentially* (so once a planet runs out, later over-budget fleets in the list send nothing). A fleet is *committed* when its fraction clears the activation threshold $\phi_{e,k} \geq \tau_{\text{act}}$ but only executes when $n_{e,k} \geq 1$ and ships remain; committed fleets that fail to execute — on phantom slots, unowned planets, or past the ship budget — are exactly the x_t counted by the penalty (Section 5).

5 Reward and return

Production-only dense reward + invalid-dispatch penalty. The shaped reward rewards *growing one's own production efficiently*, and nothing else — not raw planet count, not ship count (both of which an agent can maximize while still losing the game). Let $\Pi_0(s) = \sum_{p:\,\text{owner}(p)=0} \text{prod}(p)$ be ego's total planetary production, read directly from the state. Because the continuous actor is *not* masked (Section 3), the only other term is a penalty on *invalid* fleet dispatches. Let x_t be the number of *committed* fleets ($\phi_{e,k} \geq \tau_{\text{act}}$) that do not execute — a single count that covers, "everywhere":

- a fleet committed on a **non-existent** slot ($m_e = 0$; the target output there is $0/0$);
- a fleet committed from a planet the ego **does not own** or that has 0 ships ($u_e = 0$);
- an **over-dispatch** — a committed fleet beyond the planet's remaining ships (the K fractions summing past 100%), which the engine silently drops.

With w_Π (`prod_reward_weight`), ρ (`illegal_launch_penalty`), w_e (`enemy_growth_weight`), and scale/clip σ, κ,

$$r_t = \mathrm{clip}\left(\frac{w_\Pi\, \Delta\Pi_0(t) \;-\; \rho\, x_t \;-\; w_e\, y_t\, \mathbf{1}[\, t \geq x_0\,]}{\sigma},\, -\kappa,\, \kappa \right), \qquad \Delta\Pi_0(t) = \Pi_0(s_{t+1}) - \Pi_0(s_t). \tag{11}$$

The first two terms are as before: capturing a high-production planet is rewarded *exactly* through $\Delta\Pi_0$ (expansion toward production is the positive signal), while $\rho\, x_t$ is the sole pressure teaching the legal dynamic action space (which slots exist, which are owned, how many ships are available across the K fleets). There is no shaping on *where* a valid launch is aimed. The third term — defined next — rewards *suppressing the opponent*.

Enemy-suppression term (relative to the enemy's own recent growth). Let $\Pi_1(s) = \sum_{p:\,\mathrm{owner}(p)=1} \mathrm{prod}(p)$ be the opponent's production and $g_t = \Pi_1(s_{t+1}) - \Pi_1(s_t)$ its per-step growth. We compare it to the enemy's *own recent* growth through an exponential moving average $\bar{g}_t = (1-\beta)\bar{g}_{t-1} + \beta\, g_t$ (window set by β; $\beta = 1$ recovers the one-step second difference $g_t - g_{t-1}$):

$$y_t = g_t - \bar{g}_{t-1} \qquad \Longrightarrow \qquad -\,w_e\, y_t \begin{cases} < 0, & \text{enemy growing } \textit{faster} \text{ than before } \Rightarrow \text{ penalty,} \\ > 0, & \text{enemy growing } \textit{slower} \text{ than before } \Rightarrow \text{ reward.} \end{cases} \tag{12}$$

So the agent is rewarded for *bending the enemy's growth curve down* (raiding their economy, taking their planets) and penalized when the enemy accelerates. The indicator $\mathbf{1}[t \geq x_0]$ disables the term for the first x_0 steps (`enemy_growth_warmup`), where both players are expanding freely into neutral space and the enemy's growth is not yet attributable to the agent. Like every dense term it enters D_i below, so it is forfeited on a loss too.

Outcome, the draw anchor, and the loss forfeit. At termination the game is scored by total ships; $o_i = +1$ win, 0 draw, -1 loss. Let $D_i = \sum_t \gamma^t r_{i,t}$ be the accumulated discounted dense reward. We anchor the return on a **non-negative outcome ladder** — a draw is a *stable* 1, a win one rung above, a loss one rung below — and let the dense reward modulate it only on *decisive* games: **added on a win**, **forfeited (negated) on a loss**, with a draw flat. So a trajectory that hoarded production yet still lost is driven *negative* (w_d =`dense_weight`, w_o =`outcome_weight`):

$$\boxed{R_i = \begin{cases} 2\, w_\mathrm{o} + w_\mathrm{d} D_i, & o_i > 0 \text{ (win)}, \\ 1\, w_\mathrm{o}, & o_i = 0 \text{ (draw, stable)}, \\ 0\, w_\mathrm{o} - w_\mathrm{d} D_i, & o_i < 0 \text{ (loss)}. \end{cases}} \tag{13}$$

With the defaults $w_\mathrm{o} = w_\mathrm{d} = 1$: a draw scores exactly 1, a win $2 + D_i$, a loss $-D_i$. The base ladder $\{0, 1, 2\}$ makes *losing only slightly worse than drawing* (and drawing slightly worse than winning), while the forfeit ensures that among losses the *more* a trajectory built without converting it to a win, the worse it scores — directly penalizing "maximize my own production yet lose," which earlier shaped rewards failed to discourage. Setting `loss_forfeit`=false recovers a plain additive $R_i = w_\mathrm{o}(o_i + 1) + w_\mathrm{d} D_i$.

Implemented bounded variant (current default). The shipped reward (`RolloutConfig`, used identically by the CPU and the GPU rollouts of Section 9) replaces the loss-forfeit return above with a *smooth, bounded* per-game form: each channel is accumulated with the discount over

the episode and then capped, and the outcome is an *explicit fixed bonus* rather than a forfeit, so a single decisive game can no longer swing the return arbitrarily far. With per-game channels

$$P = \mathrm{clip}\Big(\textstyle\sum_t \delta_\Pi^t\,\gamma^t\,w_\Pi\,\Delta\Pi_0(t),\ -P_{\max},\ P_{\max}\Big), \qquad P_{\max} = \texttt{prod\_reward}$$

$$V = \mathrm{clip}\Big(\textstyle\sum_t \gamma^t\,w_{\mathrm{v}}\,v_t,\ 0,\ V_{\max}\Big), \qquad V_{\max} = \texttt{valid\_reward}$$

$$I = \textstyle\sum_t \gamma^t\,\rho\,x_t, \quad M = \sum_t \gamma^t\,\rho_{\mathrm{m}}\,m_t \ \text{(both uncapped)}, \quad S = -\sum_{t\ge x_0} \gamma^t\,w_{\mathrm{e}}\,y_t \ \text{(stage} \ge 2),$$

the return is

$$R_i = O_i + P_i + V_i - I_i - M_i + S_i, \qquad O_i = \begin{cases} +\,\texttt{win\_bonus}\,(300), & \text{win}, \\ -\,\texttt{loss\_penalty}\,(100), & \text{loss}, \\ 0, & \text{draw}, \end{cases} \tag{14}$$

where O_i is applied **only at stage** ≥ 2 (Stage 1 is pure $P+V-I-M+S$, no win/loss signal). The production term carries a per-step decay δ_Π^t (`prod_reward_decay`, default 0.997): step t's production gain is weighted by δ_Π^t, so early expansion is worth more than late farming and production's share of the return shrinks over the episode — leaving the aiming (V, M) and outcome (O) terms to drive the group-relative advantage rather than a flat, ever-growing P. Here v_t is the number of *committed* ego launches whose straight-line heading actually *lands on a planet* — the same closed-form ray-vs-disk projection (with sun occlusion) used for the threat features in Section 2, evaluated at the launch point $p+\hat{\mathbf{d}}(r_p+0.1)$. This V term is a deliberate, bounded *gradient on aiming* and so *supersedes* the "no shaping on where a launch is aimed" statement above: in practice it was needed to escape the passive basin (the policy must discover that raising ϕ on owned planets toward a real target is rewarded, not merely penalized for illegality).

Miss penalty M — the aiming *stick* (`miss_launch_penalty`). The carrot V alone proved insufficient: with the per-launch hit reward saturating at $V_{\max}$ and a *free* miss, the reward-optimal policy is to **spray** — commit on many slots, collect the few lucky landings, and pay nothing for the ships thrown into empty space (a legal, in-budget launch from an owned planet that lands on no planet *executes*, so it is *not* an invalid dispatch x_t and carries no penalty). We therefore add a symmetric penalty on the *miss* count

$$m_t = \big(\text{executed ego launches at } t\big) - v_t \ = \ \text{(committed, legal, in-budget)} - \text{(landed on a planet)}, \tag{15}$$

i.e. launches from owned planets whose straight-line heading (same ray-vs-disk projection as v_t) hits nothing. With weight ρ_{m} (`miss_launch_penalty`), $M = \sum_t \gamma^t \rho_{\mathrm{m}} m_t$ enters the return as $-M$. Now a launch is $+w_{\mathrm{v}}$ if it lands and $-\rho_{\mathrm{m}}$ if it misses, so the expected value of committing a fleet is positive only once its aim is accurate enough ($p_{\mathrm{hit}} > \rho_{\mathrm{m}}/(w_{\mathrm{v}}+\rho_{\mathrm{m}})$): spraying is no longer free, and the policy is pushed to aim rather than flood. **Guardrail (passivity):** historically a large dispatch penalty collapsed the policy to "do nothing," because under the old *scalar-angle* head precise aiming was *unrepresentable* — stopping was the only escape. Under the direction-vector action (Section 3: $\theta = \mathrm{atan2}(2d_y-1,\ 2d_x-1)$, linear in relative position and attention-computable) aiming *is* reachable, so the miss penalty drives *precision* instead of paralysis — provided $\rho_{\mathrm{m}} \le w_{\mathrm{v}}$ (a launcher that lands >half its fleets stays net-positive). It defaults to $\rho_{\mathrm{m}} = 0.01$.

Note — 2p vs. 4p objective. The margin potential Φ and the outcome o_i are the two-player ($N = 2$) zero-sum forms. In $N = 4$ free-for-all the objective is *ordinal* — be the maximum of four —

which is not zero-sum: e.g. Φ would become $\text{ships}_0 - \max_{k\neq 0} \text{ships}_k$ (margin over the *strongest* rival, or a soft-max thereof) and o_i the FFA win/lose of Section 1. The GRPO machinery (advantage, surrogate, KL) is unchanged — only R_i in Eq. (13) is redefined.

6 Curriculum

From-scratch and BC-finetuned RL repeatedly plateaued at ∼random level *versus* the scripted `starter` ($\leq$ 2% win), because two skills are entangled in the full adversarial task: (i) *aim + expand* — launch ships that actually capture planets — and (ii) *defend* — hold planets against incoming fleets. We decouple them with a three-stage curriculum (`RolloutConfig::stage`); each stage warm-starts from the previous stage's policy, which also becomes the new KL reference π_{ref} of Section 7.4 (anchor-and-advance, rather than anchoring forever to the passive BC). *All stages share the one reward of Section 5* (production-only dense + invalid-dispatch penalty, loss-forfeit return); only the **opponent** and the discount γ change across stages.

Stage 1 — solo expansion (passive opponent). The opponent issues no launches, so the policy can freely learn to aim and to take the whole map without adversarial punishment. Because the passive opponent hoards production on its home planet, the ego must actually *conquer* it (not merely grab neutrals) to win the terminal ship score — failing that, the loss forfeit (Eq. (13)) turns the episode negative. Running with $\gamma < 1$ ($\gamma = 0.99$) makes *earlier* production gains worth more, i.e. a discounted "planet-takeover-time" incentive for fastest full-map conquest; the episode ends early once the ego owns everything.

Stage 2 — 1v1 vs. starter. The opponent is always `starter` and $\gamma = 1$ (timing is now dictated by the opponent, not a solo clock). The same reward applies, so the loss forfeit is fully in force: the agent must convert its production into an actual win and must *defend* (using the threat features of Section 2) or forfeit everything it built. The Stage-1 policy supplies both the initialization and π_{ref}, so it keeps its aiming/expansion skill while learning to win.

Stage 3 — mixed opponents. The opponent per group is sampled from {`random`, `starter`, (league snapshots)} as in Section 8, generalizing beyond a single fixed adversary toward the multi-opponent FFA setting.

7 GRPO update

GRPO replaces the value baseline of actor–critic methods with a *group-relative* baseline: the policy plays a group of trajectories from the *same* start against the *same* opponent, and each trajectory's return is centered against its group.

7.1 Grouped sampling

For iteration we draw N groups (`num_groups`); group j fixes an initial state $s_0^{(j)}$ (a world) and an opponent, and rolls out G trajectories (`group_size`) under the current behavior policy $\pi_{\theta_{\text{old}}}$ (stochastic). The only within-group variation is the policy's own action sampling, so the group mean is a valid, low-variance baseline. The env count is $N \cdot G$ (`num_envs`). For every transition we store (X, m, u, g), the action $a_{i,t}$, the behavior log-prob $\log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})$, and the reference log-prob $\log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t})$ (Section 7.4).

7.2 Group-relative advantage (no critic)

Let $\mathcal{G}_j$ be the trajectories of group j, with mean and standard deviation

$$\mu_j = \frac{1}{G}\sum_{i\in\mathcal{G}_j} R_i, \qquad \varsigma_j = \sqrt{\frac{1}{G}\sum_{i\in\mathcal{G}_j}(R_i-\mu_j)^2}. \tag{16}$$

The advantage of trajectory i (group $j = j(i)$), *broadcast to all of its timesteps*, is

$$\boxed{\hat{A}_i = \begin{cases} \dfrac{R_i-\mu_j}{\varsigma_j+\varepsilon}, & \text{whiten\_advantage} = \text{true}, \\ R_i-\mu_j, & \text{otherwise}, \end{cases}} \tag{17}$$

with floor $\varepsilon =$`adv_eps`. There is no learned $V(s)$ and no temporal-difference or GAE term — the entire baseline is Eq. (17).

7.3 Clipped policy surrogate

With the per-step importance ratio

$$\rho_{i,t}(\theta) = \exp\Big(\log \pi_\theta(a_{i,t} \mid s_{i,t}) - \log \pi_{\theta_{\text{old}}}(a_{i,t} \mid s_{i,t})\Big), \tag{18}$$

(each log-prob the entity sum of Eq. (8)), the surrogate is the PPO clip objective

$$\mathcal{L}^{\text{clip}}(\theta) = -\frac{1}{\sum_i T_i}\sum_{i,t} \min\Big(\rho_{i,t}\,\hat{A}_i,\ \ \text{clip}(\rho_{i,t},\,1-\epsilon,\,1+\epsilon)\,\hat{A}_i\Big), \tag{19}$$

with clip range $\epsilon =$`clip`.

7.4 KL anchor to a reference policy

A reference policy π_{ref} (the fixed behavior-cloning checkpoint) anchors training, which counteracts the empirically observed drift of plain PPO away from the strong BC behavior. We penalize the unbiased, non-negative k_3 estimator of $\text{KL}(\pi_\theta \,\|\, \pi_{\text{ref}})$. With $\Delta_{i,t} = \log \pi_{\text{ref}}(a_{i,t} \mid s_{i,t}) - \log \pi_\theta(a_{i,t} \mid s_{i,t})$,

$$\widehat{\text{KL}}_{i,t} = \exp(\Delta_{i,t}) - \Delta_{i,t} - 1 \ \ (\geq 0), \qquad \mathcal{L}^{\text{KL}}(\theta) = \frac{1}{\sum_i T_i}\sum_{i,t} \widehat{\text{KL}}_{i,t}. \tag{20}$$

7.5 Total objective

Combining the surrogate, the KL anchor (weight $\beta =$`kl_beta`) and an entropy bonus (weight $c_{\mathcal{H}} =$`ent_coef`, from Eq. (9)):

$$\boxed{\mathcal{L}(\theta) = \mathcal{L}^{\text{clip}}(\theta) \ + \ \beta\,\mathcal{L}^{\text{KL}}(\theta) \ - \ c_{\mathcal{H}}\,\overline{\mathcal{H}}\big[\pi_\theta\big]} \tag{21}$$

where $\overline{\mathcal{H}}$ is the mean entropy over transitions.

7.6 Optimization

For each iteration: collect the grouped trajectories under $\pi_{\theta_{\text{old}}} = \pi_\theta$; compute returns (13) and advantages (17); then for `update_epochs` passes over `minibatches` shuffled minibatches, take Adam steps on (21) with learning rate η =`lr`, ϵ_{Adam} =`adam_eps`, and global gradient-norm clip `max_grad_norm`. Advantages are computed once per iteration (off $\pi_{\theta_{\text{old}}}$); the ratio (19) and KL (20) are recomputed each minibatch under the live π_θ.

8 Self-play league

Once training has reached `start_step`, the opponent for each group is sampled from a mix of {`random`, `starter`, league} with weights (`mix_random`, `mix_starter`, `mix_pool`). The league is a bounded set (`pool_capacity`) of *historical* frozen snapshots — not a single latest self — which avoids the cycling/forgetting of single-snapshot self-play. Promotion is *win-gated*: a new snapshot is admitted only when the current policy beats the pool with win rate $\geq$ `promote_winrate`, so the policy never trains against a regression of itself.

9 Batched GPU simulator

The naive rollout is *latency-bound*: each of the $T{\approx}500$ ticks is a strict chain encode (CPU) $\rightarrow$ forward (GPU) $\rightarrow$ decode+step (CPU) with a host $\leftrightarrow$ device transfer on either side of the forward, so the GPU sits idle $\sim$70% of the time waiting on the CPU simulation. `GpuEnv` (`native/rl/gpu_env.{hpp,cpp}`, enabled by `RolloutConfig::gpu_env`) instead runs the *entire* game as batched LibTorch tensor ops on the GPU, so the whole loop — encode, sample, decode, step, scripted opponent — stays on-device. The deep-policy forward and the swept fleet/planet collision then dominate, making the loop **GPU-compute-bound** (observed 100% SM utilization), not CPU- or transfer-bound. It uses *pure* tensor ops (no custom CUDA kernel / toolkit).

Layout (Structure-of-Arrays, leading batch B). Planets live in E_c = `planet_cap` *fixed* slots (the last 4 reserved for the at-most-one live comet group); fleets in a fixed pool of `fleet_cap` slots with an *alive* mask. Because the per-planet actor is permutation-equivariant (Section 3), **no entity_order sort is needed** — row order is irrelevant, which removes the one operation that does not vectorize cleanly. So here $E = E_c$ (the obs/action dim), independent of the serving-time top-40 cap. Integer quantities (ships, production, owner, step) are kept in `float32` (exact $< 2^{24}$).

Vectorized dynamics. The per-tick update mirrors the reference engine (`core/sim.hpp`) branch-free: orbital motion and production are elementwise; launches scatter into free fleet slots; the swept ray/disk collision is the $B \times$ `fleet_cap` $\times E_c$ pairwise test (the heavy on-device compute). *Combat needs no sort*: with exactly two players, arriving ships scatter-add per owner into (B, E_c), then top/second/survivor are elementwise (the reference's stable-sort/first-seen order only matters for $>$2 owners). Comets are a baked per-env schedule (spawn step, ships, precomputed path positions) overlaid each tick. The threat encoder (Section 2) breaks equal-ship ties by a per-fleet *launch-order* stamp to match the reference's `stable_sort` over the fleet list.

Parity. `GpuEnv` is a third engine and must implement the same rules as the C++ `ow::step` that is also the Kaggle submission engine. `apps/parity_gpu` drives both from identical states/actions and compares per-env aggregates: the deterministic core is **bit-exact** (64 envs $\times$ many ticks), the

observation encoder is parity-exact (max feature diff $\sim 10^{-6}$), the `starter` opponent is bit-exact, and full episodes with comets stay $\geq 90\%$ exact, the residual being benign `float32`-vs-`float64` boundary flips in marginal collisions (a training-time fidelity choice, not a logic error).

On-device rollout, host-offloaded trajectory. `GroupedRollout::collect_gpu` keeps the loop on-device but copies each step's (obs, action, log-prob) to *pinned* host buffers with non-blocking transfers, so VRAM holds only the model, the current step's working set, and the collision intermediate — bounded in T and B, i.e. **not memory-bound**. The reward channels (Section 5, bounded variant), terminal test, and group advantage are computed on-device; valid transitions then flatten into the same host `TrajectoryBatch` the CPU path produces, so the GRPO update (Section 7) is unchanged and streams minibatches back to the GPU. Within a group all envs share one (world, opponent) so the group baseline stays valid; the opponent is uniform across the batch per iteration.

Deep-net numerical stability. Summing the squashed-Gaussian log-prob over $E \cdot 3K$ components makes a deep, from-scratch policy prone to blow-ups: an extreme μ with a saturated sample (latent z far from μ) over a tiny σ^2 produces a huge `logN`, hence an exploding step and $\exp(\text{logp} - \text{old}) \to \infty$. The trainer guards this with a tight log-std range $[\varsigma_{\min}, \varsigma_{\max}] = [-2, 1]$, a clamp on μ inside the density (the saturated region is unchanged), a clamp on the log-ratio before exp, a $[-10, 10]$ clamp on the whitened advantage (a near-degenerate group otherwise explodes it), and skipping any optimizer step whose gradient norm is non-finite.

10 Symbols ↔ `config.hpp`

Symbol	`config.hpp` field	Default
d	`ModelConfig::hidden`	128
E	`ModelConfig::max_entities`	40
K	`ModelConfig::fleets_per_planet`	5
τ_{act}	`ModelConfig::act_threshold` (fleet-commit)	0.05
`use_glu`	`ModelConfig::use_glu` (trunk A/B), n_{res}	true, 2
$[\varsigma_{\min}, \varsigma_{\max}]$	`ModelConfig` log-std clip	$[-2,\ 1]$
`std_state_dependent`	`ModelConfig` (ς head vs vector)	true
G, N	`GrpoConfig::group_size`, `num_groups`	8, 64
ϵ	`GrpoConfig::clip`	0.2
β	`GrpoConfig::kl_beta`	0.04
$c_{\mathcal{H}}$	`GrpoConfig::ent_coef`	0.01
ε	`GrpoConfig::adv_eps`	10^{-4}
w_{d}, w_{o}	`GrpoConfig::dense_weight`, `outcome_weight`	1, 1
γ	`GrpoConfig::gamma`	1.0
w_Π, $P_{\max}$	`prod_reward_weight`, `prod_reward_cap`	2.5, 100
δ_Π	`prod_reward_decay` (per-step prod decay)	0.997
w_{v}, $V_{\max}$	`valid_launch_reward`, `valid_reward_cap`	0.02, 30
ρ	`illegal_launch_penalty` (uncapped I)	0.005
ρ_{m}	`miss_launch_penalty` (uncapped M, aiming stick)	0.01
win/loss	`win_bonus`, `loss_penalty` (stage ≥ 2)	300, 100
w_{e}	`enemy_growth_weight`	0.5
x_0, β	`enemy_growth_warmup`, `enemy_growth_ema`	50, 0.1
stage	`RolloutConfig::stage`	2
`gpu_env`	`RolloutConfig` (on-device rollout, Section 9)	false
E_c	`RolloutConfig::planet_cap`, `fleet_cap`	48, 1024
`N_SOON`, `N_BIG`	per-planet inbound fleets: soonest / largest (`encode.hpp`)	2, 5
d_g	board-globals embedding dim (`ModelConfig`)	32
η	`OptimConfig::lr`	3×10^{-4}

A Appendix: the PPO/GAE method being replaced

The previous trainer (`native/rl/trainer.hpp`) used actor–critic PPO with a value head V_ϕ and Generalized Advantage Estimation. For completeness, with TD residual $\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t)$,

$$\hat{A}_t^{\text{GAE}} = \sum_{l\geq 0} (\gamma\lambda)^l\, \delta_{t+l}, \qquad \mathcal{L}^V = \tfrac{1}{2}\left(V_\phi(s_t) - \hat{R}_t\right)^2, \quad \hat{R}_t = \hat{A}_t^{\text{GAE}} + V_\phi(s_t), \tag{22}$$

and total loss $\mathcal{L}^{\text{clip}} + c_V\,\mathcal{L}^V - c_{\mathcal{H}}\overline{\mathcal{H}}$. GRPO removes V_ϕ (and thus λ, c_V, and value warmup) entirely, replacing $\hat{A}_t^{\text{GAE}}$ with the group-relative $\hat{A}_i$ of Eq. (17).